Software Requirements Specification

Eagle Learning Management System (Eagle LMS)

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional, non-functional, and system-level requirements for the Eagle Learning Management System (Eagle LMS). The system is developed for Eagle Industrial Services Pvt. Ltd., a leading security and facility management company headquartered in Pune, Maharashtra, India.

The document is intended for use by:

- Development and QA teams building and testing the system
- Project managers and stakeholders reviewing scope and deliverables
- System administrators responsible for deployment and maintenance
- Trainers and trainees who are end-users of the platform

1.2 Scope

Eagle LMS is a full-stack training and compliance management platform that serves the workforce of Eagle Industrial Services Pvt. Ltd. — comprising over 2,500 employees across 110+ clients. The system provides:

- A web application (React + Vite) for both trainers and trainees
- A mobile application (Expo / React Native) for Android and iOS
- A RESTful API backend (Python FastAPI) connecting both clients to a MySQL database

The platform enables trainers to author modules, schedule live or virtual sessions, upload materials, create assessments, and track trainee performance. Trainees can browse enrolled modules, consume learning content, take timed assessments, and monitor their progress — all through a phase-gated content delivery model.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
LMS	Learning Management System
SRS	Software Requirements Specification
API	Application Programming Interface
JWT	JSON Web Token — used for stateless authentication
RBAC	Role-Based Access Control
Module	A training course unit consisting of chapters, materials, and tests
Phase	Lifecycle stage of a module: Pre-session, Live, or Post-session
Trainer	An authorised user who creates and manages training content

Trainee	An enrolled learner who consumes training content
ORM	Object-Relational Mapper — SQLAlchemy used in this project
DXA	Document XML Architecture — unit of measurement in OOXML
PSARA	Private Security Agencies Regulation Act (India)
EAS	Eagle Industrial Services Pvt. Ltd.

1.4 Document Conventions

Requirements are uniquely identified by a prefix indicating their category (FR for Functional Requirement, NFR for Non-Functional Requirement, DR for Data Requirement) followed by a sequential number. Priority levels are: Critical, High, Medium, and Low.

1.5 References

- FastAPI Documentation — https://fastapi.tiangolo.com
- React Documentation — https://react.dev
- Expo Documentation — https://docs.expo.dev
- SQLAlchemy ORM — https://docs.sqlalchemy.org
- JSON Web Tokens (RFC 7519)
- OWASP Web Application Security Testing Guide

2. Overall Description

2.1 Product Perspective

Eagle LMS is a purpose-built, standalone application for Eagle Industrial Services Pvt. Ltd. It replaces any legacy or manual training record-keeping processes. The system comprises three tightly integrated components:

1. Backend API Server — A Python FastAPI application connected to a MySQL database, responsible for all business logic, data persistence, authentication, and file management.
2. Web Frontend — A React single-page application served via a Vite build, accessed via any modern browser on desktop or laptop devices.
3. Mobile Application — A cross-platform Expo/React Native app deployable to Android (via Google Play) and iOS (via App Store or Expo Go for internal distribution).

All three components share a single API layer. The web and mobile frontends are clients of the same REST API endpoints, ensuring data consistency.

2.2 Product Functions — High-Level Summary

The following summarises the principal capabilities of Eagle LMS:

Function Area	Summary
Authentication & Authorisation	JWT-based login and registration, role-based access for Trainer and Trainee
Module Management	Create, schedule, publish, and delete training modules with colour coding and training type
Content Delivery	Phase-gated release of materials (PDF, PPT, video, image) with per-user watermarking
Assessment Engine	Create and manage pre/mid/post-session MCQ tests with timers, scoring, and attempt limits
Progress Tracking	Track material completion and watch percentage per trainee per module
Reporting	Per-module trainee performance tables showing test scores, pass/fail status
Notifications	In-app notification delivery when modules are published or materials/tests are added
Calendar	Visual monthly calendar showing module and test events per trainee
Profile Management	User profile editing, avatar upload, and password change for trainees

2.3 User Classes and Characteristics

2.3.1 Trainer

Trainers are authorised Eagle Industrial Services staff responsible for creating and managing all training content. They have full administrative access to module creation, scheduling, content uploading, test creation, and reporting. Trainers access the system primarily via the web application.

2.3.2 Trainee

Trainees are Eagle Industrial Services employees (security guards, housekeeping staff, facility managers, etc.) enrolled in one or more training modules. They access learning materials, take assessments, and track their progress. Trainees use both the web and mobile applications. The system must accommodate users with varying levels of digital literacy.

2.4 Operating Environment

Component	Environment
Backend API	Ubuntu 24 LTS, Python 3.11+, MySQL 8.x on port 3307, served via Uvicorn on port 8000
Web Frontend	Modern browsers (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+); React 18, Vite 5
Mobile App	Android 10+ and iOS 14+; Expo SDK 54, React Native 0.81
File Storage	Local filesystem at /static/uploads on the server
Network	HTTPS preferred for production; local LAN with HTTP acceptable for internal deployment

2.5 Design and Implementation Constraints

- The backend uses SHA-256 password hashing (not bcrypt) to maintain compatibility with the existing Flask application.
- File storage is local filesystem-based, not cloud object storage.
- The mobile application detects the host IP dynamically via Expo's hostUri for API connectivity.
- The system uses a single MySQL database shared by all components.
- The web frontend communicates with the API via a Vite proxy during development; in production, both are served from the same origin or behind a reverse proxy.

2.6 Assumptions and Dependencies

- A valid MySQL 8.x instance is available and accessible at the configured DATABASE_URL.

- The server has sufficient storage for uploaded training materials (videos, PDFs, presentations).
- Trainee devices have an internet or LAN connection to the API server.
- The Expo Go app or a compiled APK/IPA is available for mobile users.
- Python packages listed in requirements.txt are installable in the target environment.

3. System Architecture

3.1 Architecture Overview

Eagle LMS follows a three-tier architecture pattern:

4. Presentation Tier — React web app and Expo mobile app serve as the user interface layer.
5. Application Tier — FastAPI backend processes all business logic, authentication, and data transformation.
6. Data Tier — MySQL database stores all persistent data; the local filesystem stores uploaded binary files.

Communication between tiers uses RESTful HTTP/HTTPS with JSON payloads. File access is secured via JWT tokens passed as query parameters for direct browser navigation.

3.2 Backend Architecture

3.2.1 Technology Stack

Component	Technology	Version
Web Framework	FastAPI	0.111.0
ASGI Server	Uvicorn (standard)	0.30.1
ORM	SQLAlchemy	2.0.30
Database Driver	PyMySQL	1.1.1
Authentication	python-jose (JWT)	3.3.0
Image Processing	Pillow	10.3.0
PDF Processing	pypdf + ReportLab	4.2.0 / 4.2.0
Environment Config	python-dotenv	1.0.1

3.2.2 Module Structure

- main.py — FastAPI application factory, CORS middleware, router registration
- database.py — SQLAlchemy engine, session factory, Base declaration
- models.py — ORM model definitions for all database tables
- schemas.py — Pydantic schemas for request validation and response serialisation
- auth.py — Password hashing, JWT creation and decoding
- deps.py — FastAPI dependency injectors (get_current_user, require_trainer)
- watermark.py — PDF and image watermarking utilities
- routers/auth.py — Login and registration endpoints
- routers/trainer.py — All trainer-facing CRUD endpoints
- routers/trainee.py — All trainee-facing read and interaction endpoints

- routers/progress.py — Material progress update endpoint
- routers/notifications.py — Notification listing and mark-read endpoints
- routers/files.py — Authenticated file serving with on-demand watermarking

3.3 Frontend Architecture (Web)

The web frontend is a single-page application built with React 18 and React Router 6. It uses Axios for API communication with a request interceptor that attaches the JWT Bearer token from localStorage. An Axios response interceptor auto-redirects to /login on HTTP 401.

- src/context/AuthContext.jsx — Global authentication state (user, login, logout, updateName)
- src/api/client.js — Axios instance with interceptors
- src/api/fileUrl.js — Utility to build authenticated file URLs with token query param
- src/components/ — Reusable Layout, Sidebar, Topbar components
- src/pages/auth/ — Login and Register pages
- src/pages/trainer/ — Dashboard, Modules, ModuleDetail, CreateTest, EditTest, Reports, Trainees
- src/pages/trainee/ — Dashboard, Module, TakeTest, TestResult, Profile, Calendar, Notifications

3.4 Mobile Architecture

The mobile application is built with Expo SDK 54 and React Native 0.81. Navigation is handled by React Navigation with a Stack Navigator wrapping role-specific Bottom Tab Navigators. JWT tokens are stored securely via expo-secure-store. The API base URL is derived dynamically from Expo's hostUri at runtime.

3.5 Data Architecture

3.5.1 Entity Relationship Summary

Entity	Relationship	Related Entity
User	one-to-many	Module (trainer_id), Enrollment (trainee_id), Notification (user_id), TestAttempt (trainee_id)
Module	one-to-many	Chapter, Material, Test, Enrollment
Chapter	one-to-many	Material (chapter_id)
Test	one-to-many	Question, TestAttempt
TestAttempt	many-to-one	Test, User (trainee)
Progress	many-to-one	Module, User (trainee), Material
Enrollment	many-to-many	Module, User (trainee) — unique constraint

4. Functional Requirements

4.1 Authentication and Authorisation

4.1.1 User Login

ID	FR-AUTH-001
Title	User Login
Priority	Critical
Description	The system shall allow users to authenticate using an email address and password. On successful authentication, a JWT access token (HS256, 1440-minute expiry) shall be returned along with the user role, name, and ID.
Input	Email (string), Password (string)
Output	access_token, token_type, role, name, user_id
Error Cases	Invalid credentials: HTTP 401 with message 'Invalid email or password'

4.1.2 User Registration

ID	FR-AUTH-002
Title	User Registration
Priority	Critical
Description	The system shall allow new users to self-register with a name, email, password, optional phone, optional department, and role (trainee or trainer). Passwords shall be stored as SHA-256 hashes. On registration, the new trainee shall be automatically enrolled in all currently published modules.
Error Cases	Duplicate email: HTTP 400 with message 'Email already registered'

4.1.3 Role-Based Access Control

ID	FR-AUTH-003
Title	Role-Based Access Control
Priority	Critical
Description	All API routes except /api/auth/login and /api/auth/register shall require a valid Bearer JWT. Routes under /api/trainer/* shall additionally require the authenticated user to have role='trainer'. Unauthorised access shall return HTTP 401 or HTTP 403.

4.2 Module Management (Trainer)

4.2.1 Create Module

ID	FR-MOD-001
Priority	Critical
Description	Trainers shall be able to create a new training module by providing a title, optional description, optional category, and a list of trainee IDs to enrol. The module is created with status='draft'.

4.2.2 Schedule Module

ID	FR-MOD-002
Priority	Critical
Description	Trainers shall be able to set start_datetime, end_datetime, status (draft/published), colour, training_type (self_paced/virtual/classroom), and meet_link for a module. When status changes from draft to published, in-app notifications shall be dispatched to all trainees.

4.2.3 Module Phase Determination

ID	FR-MOD-003
Priority	Critical
Description	The system shall compute a module's current phase at runtime based on server time relative to start_datetime and end_datetime: (1) now < start_datetime => 'pre'; (2) start_datetime <= now <= end_datetime => 'live'; (3) now > end_datetime => 'post'. If start_datetime is null, the phase defaults to 'pre'.

4.2.4 Delete Module

ID	FR-MOD-004
Priority	High
Description	Trainers shall be able to delete a module. Cascade deletion shall remove all associated chapters, materials (including files from disk), tests, questions, test attempts, enrollments, and progress records.

4.3 Content Management (Trainer)

4.3.1 Chapter Management

ID	FR-CONT-001

Priority	High
Description	Trainers shall be able to add chapters to a module with a title and automatically assigned order_num. Chapters can be deleted; deletion cascades to all materials within the chapter.

4.3.2 Material Upload

ID	FR-CONT-002
Priority	Critical
Description	Trainers shall be able to upload files in the following formats: PDF, PPT, PPTX, MP4, MOV, AVI, MKV, WEBM, PNG, JPG, JPEG. Each material shall have a title, release phase (pre/live/post), and optional chapter assignment. Uploaded files are stored on the server filesystem. An in-app notification is dispatched to all trainees on upload.
Constraints	Allowed extensions: pdf, ppt, pptx, mp4, mov, avi, mkv, webm, png, jpg, jpeg. File type detection is based on extension.

4.3.3 Phase-Gated Content Delivery

ID	FR-CONT-003
Priority	Critical
Description	A material is accessible to a trainee only when the module's current phase is equal to or later than the material's release_phase. The ordering is: pre (1) <= live (2) <= post (3). Materials with phase 'live' are not accessible during the 'pre' phase.

4.3.4 Per-User Content Watermarking

ID	FR-CONT-004
Priority	High
Description	When a trainee accesses a file of type PDF, PNG, JPG, JPEG, or WEBP, the system shall generate a watermarked copy containing the text 'Eagle Securities | <user_email>' and cache it as wm_<user_id>_<filename>. Trainers receive the original file without watermarking.

4.4 Assessment Engine

4.4.1 Test Creation

ID	FR-TEST-001
Priority	Critical

Description	Trainers shall be able to create MCQ tests linked to a module with: title, test_type (pre/mid/post), duration_minutes, start_datetime, end_datetime, passing_marks (percentage), max_attempts, and a set of questions each with text, four options (A–D), correct answer, and marks value.

4.4.2 Test Access Control

ID	FR-TEST-002
Priority	Critical
Description	Test access shall enforce: (1) test window (start_datetime <= now <= end_datetime); (2) module phase rules — pre-test available in pre/live phases; mid-test only in live phase; post-test in live/post phases; (3) attempt limit — access denied if attempt count >= max_attempts.

4.4.3 Test Submission and Scoring

ID	FR-TEST-003
Priority	Critical
Description	On test submission, the system shall: calculate score as sum of marks for correctly answered questions; compute percentage = (score / total_marks) * 100; determine passed = percentage >= passing_marks; persist a TestAttempt record with all answer data (JSON), score, percentage, passed flag, and submitted_at timestamp. Return full results including correct answers for the review screen.

4.4.4 Test Timer

ID	FR-TEST-004
Priority	High
Description	The web and mobile clients shall display a countdown timer initialised to duration_minutes * 60 seconds. The timer shall visually indicate warning state (<=3 minutes remaining) and urgent state (<=1 minute remaining). Auto-submission shall trigger on timer expiry.

4.5 Progress Tracking

ID	FR-PROG-001
Priority	High
Description	The system shall record a Progress entry per (module, trainee, material) triple, storing completed (boolean) and watch_percent (0–100). A video is considered complete when watch percentage exceeds 90%. A document is marked complete

	on explicit user action. Overall module progress percentage = (completed_materials / total_materials) * 100.

4.6 Notification System

ID	FR-NOTIF-001
Priority	Medium
Description	The system shall create in-app Notification records for all trainees when: (1) a module is published (type: module_published); (2) a material is uploaded (type: material_upload); (3) a test is created (type: test_created). Notifications have is_read (default false), and are marked read in bulk when the notifications list is fetched.

4.7 Reporting (Trainer)

ID	FR-REP-001
Priority	High
Description	For any module, trainers shall be able to view a performance report table listing all enrolled trainees with their name, email, department, completion status, and latest test attempt result (score, percentage, pass/fail) for each test in the module.

4.8 Trainee Dashboard

ID	FR-DASH-001
Priority	High
Description	The trainee dashboard shall display enrolled modules segmented into three lists: upcoming (phase='pre'), ongoing (phase='live'), and completed (phase='post'). It shall also display total tests taken, tests passed, and recent notifications. Filtering by status, training type, and keyword search shall be supported.

4.9 Trainee Profile Management

The system shall allow trainees to: (a) view and edit their name, email, phone, and department; (b) upload a profile picture (JPG/PNG/WEBP); (c) change their password by providing current and new password with confirmation. The trainer dashboard shall display a list of all registered trainees with enrollment counts and test attempt counts.

4.10 Training Calendar

ID	FR-CAL-001
Priority	Medium
Description	The system shall provide a monthly calendar view for trainees listing all enrolled module events and scheduled test events. Events shall be colour-coded by module colour. Clicking an event shall show details and navigate to the module. The web calendar shall support month navigation.

5. Non-Functional Requirements

5.1 Performance

ID	Priority	Requirement
NFR-PERF-001	High	API responses for dashboard and module listing endpoints shall complete within 2 seconds under normal load (up to 50 concurrent users).
NFR-PERF-002	High	File serving for watermarked PDFs/images shall complete within 5 seconds for files up to 20 MB. Watermarked versions shall be cached and not regenerated on repeat access.
NFR-PERF-003	Medium	The web application initial page load (LCP) shall be under 3 seconds on a standard broadband connection.
NFR-PERF-004	Medium	The mobile application screen transitions shall complete within 300 ms on mid-range Android/iOS devices.

5.2 Security

ID	Priority	Requirement
NFR-SEC-001	Critical	All API endpoints (except /api/auth/*) shall require a valid, non-expired JWT in the Authorization header.
NFR-SEC-002	Critical	File access for trainees shall require authentication via JWT query parameter. Unauthenticated file requests shall return the original file without user identification.
NFR-SEC-003	Critical	Passwords shall be stored as SHA-256 hashes. Plaintext passwords shall never be stored or logged.
NFR-SEC-004	High	The SECRET_KEY used for JWT signing shall be configurable via environment variable and shall not be hardcoded in production deployments.
NFR-SEC-005	High	CORS policy shall be configurable. In production, origins shall be restricted to known frontend domains.
NFR-SEC-006	Medium	All file uploads shall be validated against an allowlist of extensions before storage. Unsupported file types shall return HTTP 400.

5.3 Reliability and Availability

- NFR-REL-001 (High): The system shall be available 99% uptime during normal business hours (8 AM – 8 PM IST, Monday–Saturday).

- NFR-REL-002 (High): The database connection shall use connection pooling with pre-ping enabled to recover gracefully from stale connections.
- NFR-REL-003 (Medium): Failed API requests on the mobile client shall display user-friendly error messages and shall not crash the application.

5.4 Usability

- NFR-USE-001 (High): The web application shall be fully functional on screen widths from 768px (tablet) to 1920px (desktop). Responsive breakpoints shall adapt the layout at 768px and 900px.
- NFR-USE-002 (High): The mobile application shall support both Android and iOS with consistent UI behaviour. Safe area insets shall be respected on iOS.
- NFR-USE-003 (Medium): The system shall support a light/dark theme toggle on both web and mobile. Theme preference shall persist across sessions (localStorage on web, AsyncStorage/SecureStore on mobile).
- NFR-USE-004 (Medium): Loading states shall be clearly indicated with skeleton loaders or activity indicators during data fetching operations.

5.5 Maintainability

- NFR-MAINT-001 (High): Configuration parameters (database URL, JWT secret, token expiry, upload directory) shall be managed via a .env file and must not be hardcoded.
- NFR-MAINT-002 (Medium): Database schema changes shall be managed via migration scripts (e.g., migrate.py) rather than direct schema alterations.
- NFR-MAINT-003 (Medium): The seed script (seed.py) shall allow initial data population (default users and modules) in a idempotent manner.

5.6 Scalability

- NFR-SCALE-001 (Medium): The system architecture shall support horizontal scaling of the FastAPI backend by deploying multiple Uvicorn worker processes.
- NFR-SCALE-002 (Low): File storage shall be abstracted so that migration from local filesystem to a cloud object store (e.g., S3-compatible) requires minimal code changes.

6. Data Requirements

6.1 Data Models

The following table lists all persistent data entities, their key attributes, and constraints.

6.1.1 Users

Field	Type / Constraints	Description
id	Integer, PK, auto	Unique user identifier
name	String(255), NOT NULL	Full name
email	String(255), UNIQUE	Login email — indexed
password	String(64), NOT NULL	SHA-256 hex digest
role	String(20), default='trainee'	'trainer' or 'trainee'
phone	String(30), nullable	Contact phone number
department	String(255), nullable	Organisational department
profile_pic	String(255), nullable	Filename of uploaded avatar
created_at	DateTime, server default	Account creation timestamp

6.1.2 Modules

Field	Type / Constraints	Description
id	Integer, PK, auto	Unique module identifier
title	String(255), NOT NULL	Module display title
description	Text, nullable	Module overview text
category	String(100), nullable	e.g. 'Safety & Compliance'
trainer_id	FK users.id, SET NULL	Owning trainer
start_datetime	DateTime, nullable	Module start time
end_datetime	DateTime, nullable	Module end time
status	String(20), default='draft'	'draft' or 'published'

training_type	String(50), default='self_paced'	'self_paced', 'virtual', 'classroom'
color	String(10), default='#3B5BDB'	UI accent colour hex
meet_link	String(500), nullable	URL for virtual/classroom link
is_default	Boolean, default=False	Seeded default modules flag

6.1.3 Tests and Questions

Field	Type / Constraints	Description
test_type	String(10), NOT NULL	'pre', 'mid', or 'post'
duration_minutes	Integer, default=30	Test time limit
passing_marks	Integer, default=60	Minimum percentage to pass
max_attempts	Integer, default=1	Maximum allowed attempts
correct_option	String(1)	'A', 'B', 'C', or 'D'
answers	Text (JSON)	Serialised {question_id: option} map

6.2 Data Retention and Storage

- User accounts and all associated records are retained indefinitely unless manually deleted by an administrator.
- Uploaded files are stored at {UPLOAD_DIR}/{module_id}_{timestamp}_{filename}.
- Watermarked file copies are stored as wm_{user_id}_{original_filename} and are not automatically purged.
- Test attempt answers are stored as JSON strings in the answers column of test_attempts.

7. External Interface Requirements

7.1 API Interface

The backend exposes a RESTful API at the base path /api. All request and response bodies are JSON-encoded (Content-Type: application/json) except for multipart file uploads. The API is self-documented via Swagger UI at /docs and ReDoc at /redoc.

7.1.1 Authentication Endpoints

Method	Path	Auth	Description
POST	/api/auth/login	None	Authenticate user, return JWT
POST	/api/auth/register	None	Register new user, return JWT
GET	/api/auth/me	Bearer JWT	Return current user profile

7.1.2 Trainer Endpoints (selection)

Method	Path	Description
GET	/api/trainer/dashboard	Dashboard stats, modules, recent enrollments
GET	/api/trainer/modules	List all trainer modules with stats
POST	/api/trainer/module/create	Create new module with trainee enrollment
POST	/api/trainer/module/{id}/schedule	Set schedule, status, type, meet link
POST	/api/trainer/module/{id}/upload	Upload material file (multipart)
POST/PUT/DEL	/api/trainer/module/{id}/test	Create / update / delete tests
GET	/api/trainer/module/{id}/reports	Per-module trainee performance report
GET	/api/trainer/trainees	List all registered trainees

7.1.3 Trainee Endpoints (selection)

Method	Path	Description
GET	/api/trainee/dashboard	Upcoming/ongoing/completed modules, test stats
GET	/api/trainee/module/{id}	Full module detail with content, tests, progress
GET	/api/trainee/test/{id}	Fetch test questions (enforces access rules)
POST	/api/trainee/test/{id}/submit	Submit answers, get scored result
GET	/api/trainee/calendar	Calendar events (modules + tests)

GET/PUT	/api/trainee/profile	View or update trainee profile
POST	/api/progress/update	Update material completion and watch percent
GET	/uploads/{filename}	Serve file; applies watermark for trainees

7.2 Hardware Interface

- The backend runs on any x86-64 Linux server with minimum 2 CPU cores, 4 GB RAM, and 50 GB disk storage.
- The mobile application interfaces with the device camera (future) and storage for file downloads, and requires network access.

7.3 Software Interface

- MySQL 8.x database server accessible at the DATABASE_URL configured in .env.
- LibreOffice (optional, for future PDF conversion features).
- Expo Go or EAS Build for mobile distribution.

7.4 Communication Interface

- HTTP/1.1 or HTTP/2 for all API communication.
- JSON Web Token (RFC 7519) for stateless session management.
- Multipart/form-data for file upload endpoints.

8. System Constraints and Limitations

8.1 Known Constraints

- SHA-256 password hashing is used instead of bcrypt to maintain backward compatibility with an existing Flask application. This is a known security compromise and should be addressed in future iterations.
- File storage is local filesystem only. There is no cloud storage integration. Server disk space directly limits the volume of training materials.
- Watermarked file copies accumulate indefinitely. No automatic cleanup mechanism exists for stale watermarked files.
- The CORS policy is currently set to allow all origins (allow_origins=["*"]), which is acceptable for development but must be restricted in production.
- The mobile application infers the API server IP from Expo's hostUri, which requires the device and server to be on the same local network during development.
- There is no email notification system. All notifications are in-app only.
- The system does not implement soft-delete; all deletions are permanent.

8.2 Future Scope (Out of Current Version)

- Email/SMS notification integration
- Cloud file storage (AWS S3 or compatible)
- Video streaming with adaptive bitrate (HLS)
- Certificate generation on module completion
- Bulk user import via CSV/Excel
- Admin super-user role with cross-trainer visibility
- Push notifications for mobile application
- Offline mode for mobile content consumption

9. Appendices

Appendix A — Default Training Module Catalogue

The following ten modules are seeded by default via seed.py:

#	Module Title	Category	Colour
1	Security Fundamentals & Post Orders	Security Operations	#3B5BDB
2	Fire Safety & Emergency Response	Safety & Compliance	#F04438
3	Access Control & Visitor Management	Security Operations	#0BA5EC
4	CCTV Operations & Surveillance	Technology	#7C3AED
5	Patrolling Techniques & QRT Procedures	Field Operations	#F79009
6	Housekeeping Standards & Hygiene Protocols	Facility Management	#12B76A
7	Legal Aspects of Security	Legal & Compliance	#6D28D9
8	Soft Skills & Client Interaction	Professional Development	#0891B2
9	Crowd & Event Management	Field Operations	#D97706
10	First Aid & Basic Medical Response	Safety & Compliance	#059669

Appendix B — Default User Credentials (Seed Data)

Name	Email	Password	Role
Rajesh Kumar	trainer@eagle.com	trainer123	trainer
Arjun Sharma	trainee@eagle.com	trainee123	trainee
Priya Mehta	priya@eagle.com	trainee123	trainee
Mohammed Iqbal	iqbal@eagle.com	trainee123	trainee

Appendix C — Environment Variables

Variable	Default Value	Description
DATABASE_URL	mysql+pymysql://...eagle_lms	Full SQLAlchemy connection string
SECRET_KEY	(change in production)	HS256 JWT signing key

ALGORITHM	HS256	JWT signing algorithm
ACCESS_TOKEN_EXPIRE_MINUTES	1440 (24 hours)	JWT token lifetime in minutes
UPLOAD_DIR	../static/uploads	Filesystem path for uploaded files

Appendix D — Requirement Traceability Matrix

The following table maps each functional requirement to the primary source code location implementing it.

Requirement ID	Requirement Title	Implementation Location
FR-AUTH-001	User Login	backend/routers/auth.py — POST /api/auth/login
FR-AUTH-002	User Registration	backend/routers/auth.py — POST /api/auth/register
FR-AUTH-003	RBAC	backend/deps.py — get_current_user, require_trainer
FR-MOD-001	Create Module	backend/routers/trainer.py — POST /api/trainer/module/create
FR-MOD-002	Schedule Module	backend/routers/trainer.py — POST /api/trainer/module/{id}/schedule
FR-MOD-003	Phase Determination	backend/routers/trainee.py — _get_phase(); backend/routers/trainer.py — _module_phase()
FR-CONT-002	Material Upload	backend/routers/trainer.py — POST /api/trainer/module/{id}/upload
FR-CONT-003	Phase-Gated Delivery	frontend/src/pages/trainee/Module.jsx — canAccess(); mobile/src/screens/trainee/ModuleDetail.js — canAccess()
FR-CONT-004	Watermarking	backend/watermark.py; backend/routers/files.py — GET /uploads/{filename}
FR-TEST-001	Test Creation	backend/routers/trainer.py — POST /api/trainer/module/{id}/test
FR-TEST-002	Test Access Control	backend/routers/trainee.py — GET /api/trainee/test/{id}
FR-TEST-003	Test Scoring	backend/routers/trainee.py — POST /api/trainee/test/{id}/submit
FR-PROG-001	Progress Tracking	backend/routers/progress.py — POST /api/progress/update
FR-NOTIF-001	Notifications	backend/routers/trainer.py — _notify_trainees(); backend/routers/notifications.py

FR-REP-001	Reports	backend/routers/trainer.py — GET /api/trainer/module/{id}/reports
FR-CAL-001	Calendar	backend/routers/trainee.py — GET /api/trainee/calendar